Keys in DBMS

What are Keys?

- Keys are the essential elements of any relational database.
- It identifies each tuple in a relation uniquely.
- Keys are also used to establish the relationship among the tables in a schema

Whay do we need Key?

Need for keys in RDBMS

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Increase the salary of John by 10%

Here three Johns are there in my database

-Increase the salary of John by Dept Name

Again there will come two John having same Dept Name.

So, we have to identify unique tuples, i.e., why we need keys? If we use ID no as a unique key. We can update the salary of the person who is having the ID no.105

DBMS KEYS

- DBMS defines five types of Keys and they all have different functionalities

- The Keys are as follows:
  - Super Key
  - Candidate Key
  - Primary Key
  - Alternate Key
  - Unique Key
  - Composite Key
  - Foreign Key

Let's discuss one by one all of the five keys

Super Key

Need for keys in RDBMS

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Super Key

{Name, Phone No}
{ID}-SK
{Name}-it is not SK (Multiple people have same name)
-It may contain extraneous attributes
{ID, Name}-SK
{ID, SSN} –SK
{Name, SSN}-SK
{SSN}-SK
{ID, Phone}-SK
{Name, SSN, Phone}- SK
{Name, Email} – SK
{ID, SSN, Phone} -SK

Super Key

- Superset
- All possible keys that can be formed from a relation or a table
- We can create any number of key combination from this table

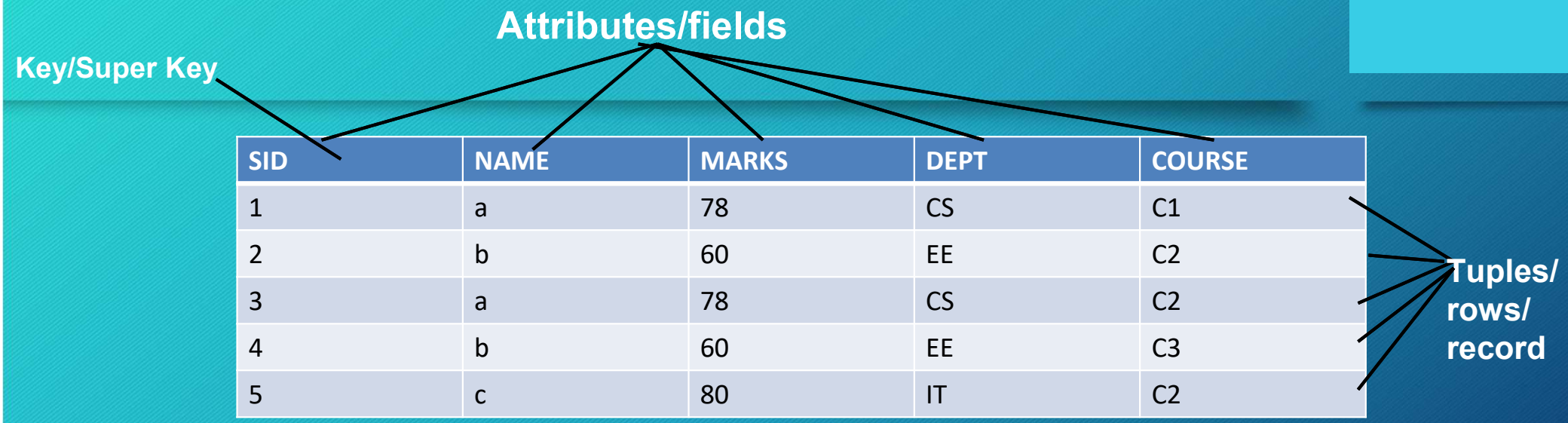
{ID}, {ID, Name}, {Name, Salary}

- Why do we need super keys? Why not other keys?
- Super key is a superset from which we can derive other keys.
- Uniquely identify the tuple
- It contains some extraneous attributes

Super Key

- A single attribute or a set of attributes that can uniquely identify all rows of a particular relation is called Super key.
- A ***Super Key*** can contains one or more than one attributes.
- A Super Key is the most general type of key.

Super Key



SID	NAME	MARKS	DEPT	COURSE
1	a	78	CS	C1
2	b	60	EE	C2
3	a	78	CS	C2
4	b	60	EE	C3
5	c	80	IT	C2

{Dept, Course} Key/Super key
CS C1 - no where this same record exist
{Name, Marks} not key/ SK
a 78 - Repeated in another record
{Name, Marks, Dept, Course} – key/SK (No two records can exactly be same, may be some attributes are same)

SK/Key- Attribute or set of attributes that uniquely identify each record of relation

Super Key

{SID} –SK
{SID, NAME} – SK
{SID, NAME, MARKS} – SK
{NAME, MARKS, DEPT} – NOT SK
{MARKS, COURSE, DEPT} – SK

Q. How many maximum number of SK is possible in a relation?
Here maximum 5 attributes (individual attribute is a SK)

$$^{n}C_{r} = \frac{n!}{r!(n-r)!}$$

5+10+10+5+1=31
If attributes are four, It will be 15
So, the formula is $2^n - 1$ (maximum SK possible)
n- number of attributes

Super Key

R(A, B, C, D)

A	B	C	D
1	1	5	1
2	1	7	1
3	1	7	1
4	2	7	1
5	2	5	1
6	2	5	2

Maximum SK= $2^4 - 1 = 15$
{A} –SK (UNIQUE)
{AB} ,{AC}, {AD}, {ABC}, {ACD}, {ABD}, {ABCD}
{ADC} – it can be a SK irrespective of any order
{B}. {C}, {D}- NOT SK
{BC}, {BCD}-NOT A SK (SUBSET ALSO CAN NOT BE A SK)
TOTAL SK=8

Super Key Example

- Let's take an example to understand this: Employee table

Emp_ID	Emp_Number	Emp_Name
E01	226	Steve
E02	227	Ajeet
E03	228	Chaitanya
E04	229	Robert

Super keys:

{Emp_ID}
{Emp_Number}
{Emp_ID, Emp_Number}
{Emp_ID, Emp_Name}
{Emp_ID, Emp_Number, Emp_Name}
{Emp_Number, Emp_Name}

All of the above sets are able to uniquely identify rows of the employee table.

Candidate Key

- A candidate key is a super key that contains no extra attribute.
- Candidate keys are selected from the set of super keys, the only thing we take care while selecting candidate key is: It should not have any redundant attributes.
- Candidate key is also called minimal super key.

Candidate Key Example

- Let's take an example to understand this: Employee table

Emp_Id	Emp_Number	Emp_Name
E01	226	Steve
E12	227	Ajeet
E22	228	Chaitanya
E32	229	Robert

There are two candidate keys in above table:
{Emp_Id}
{Emp_Number}

Note: A primary key is being selected from the group of candidate keys. That means we can either have Emp_Id or Emp_Number as primary key.

Candidate Key

Need for keys in RDBMS

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

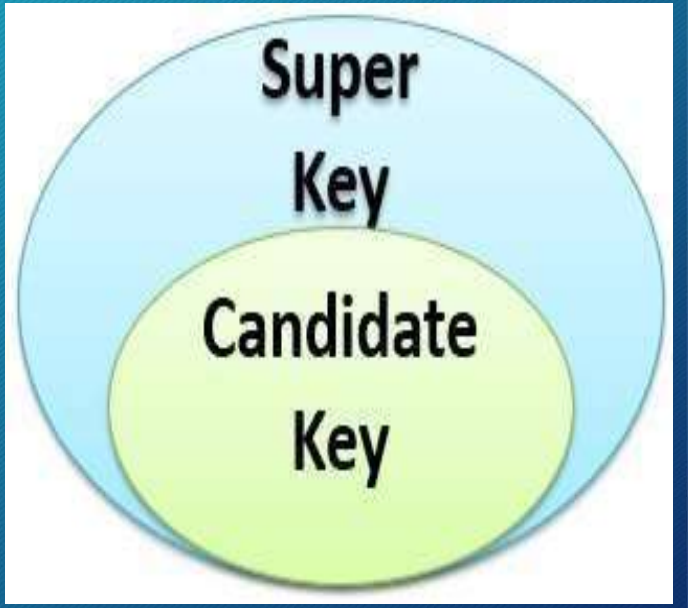
Candidate Key

- Minimal super keys are called Candidate Keys

- Super Keys (out of all SK we will find Candidate key)
- {ID}, {SSN}, {ID, NAME}, {ID, SSN}, {ID, PHONE NO}, {NAME, PHONE NO}, {ID, EMAIL}, {NAME, SSN, PHONE}, {NAME, EMAIL}, {ID, SSN, PHONE}
- {ID, NAME}-ID is SK it is not a CK
- {ID, SSN}- ID is SK it is not a CK
- {ID, PHONE NO}- ID is SK it is not a CK
- {NAME, PHONE NO}- It is a CK
- {ID, EMAIL}- ID is SK it is not a CK
- Candidate Keys
- {ID}, {SSN}, {NAME, PHONE NO}, {EMAIL}

Difference Between Super Key and Candidate Key

- A single attribute or a set of attributes that can uniquely identify all tuples of a particular relation is called Super key. On the other hands, a super key that is
  a proper subset of another super key is called candidate key.
- All candidate keys are super keys but the inverse is not true.
- The set of super keys is verified to find candidate keys whereas, the set of the candidate keys is verified to select a single primary key.



Primary key in DBMS

- A primary key is a column or set of columns in a table that uniquely identifies tuples (rows) in that table.

Example:
Student Table

Stu_Id	Stu_Name	Stu_Age
101	Steve	23
102	John	24
103	Robert	28
104	Carl	22

In the above Student table, the Stu_Id column uniquely identifies each row of the table.

Properties of Primary Keys….

- A relation can have only one primary key
- Each value in a primary key must be unique.
- Primary key cannot contain null values.

Note

- **Overall**
- ***Super Key*** is the broadest unique identifier;
- ***Candidate Key*** is a subset of Super Key;
- ***Primary Key*** is a subset of Candidate Key.

Foreign keys

- Foreign keys are the columns of a table that points to the primary key of another table

For example:

- In the below example the Stu_Id column in Course_enrollment table is a foreign key as it points to the primary key of the Student table.

Course_enrollment table:

Course_Id	Stu_Id
C01	101
C02	102
C03	101
C05	102

Student Table

Stu_Id	Stu_Name	Stu_Age
101	Chaitanya	22
102	Arya	26
103	Bran	25
104	Jon	21

Composite key

- A key that consists of more than one attribute to uniquely identify rows (also known as records & tuples) in a table is called composite key.

Example: Table - Sales

cust_Id	order_Id	product_code	product_count
C01	O001	P007	23
C02	O123	P007	19
C02	O124	P230	82
C01	O021	P890	42

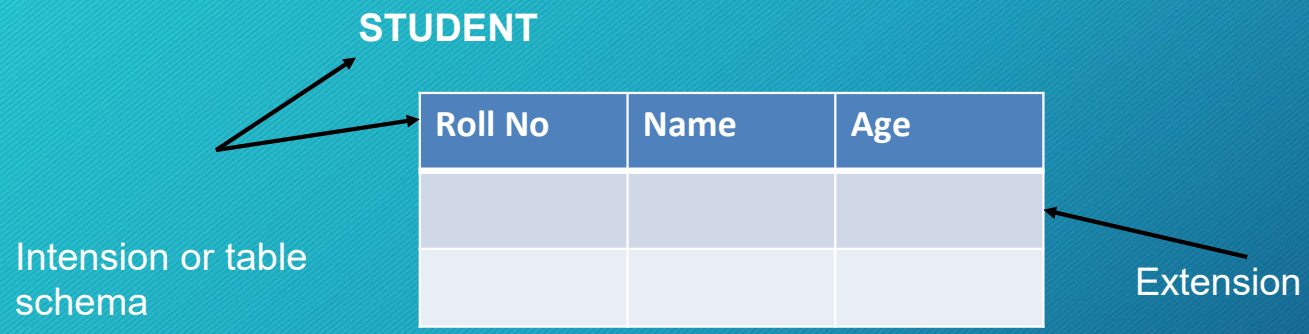
Key in above table: {cust_id, order_id}

This is a composite key as it consists of more than one attribute.

Relation Intention and Extension

Table/Relation is divided into two parts: Intension and extension

- Intension- table name + name of each of its columns
- Extension- List of all possible rows of the table



Relation Intention and Extension

Intension (Database Schema/Structure)

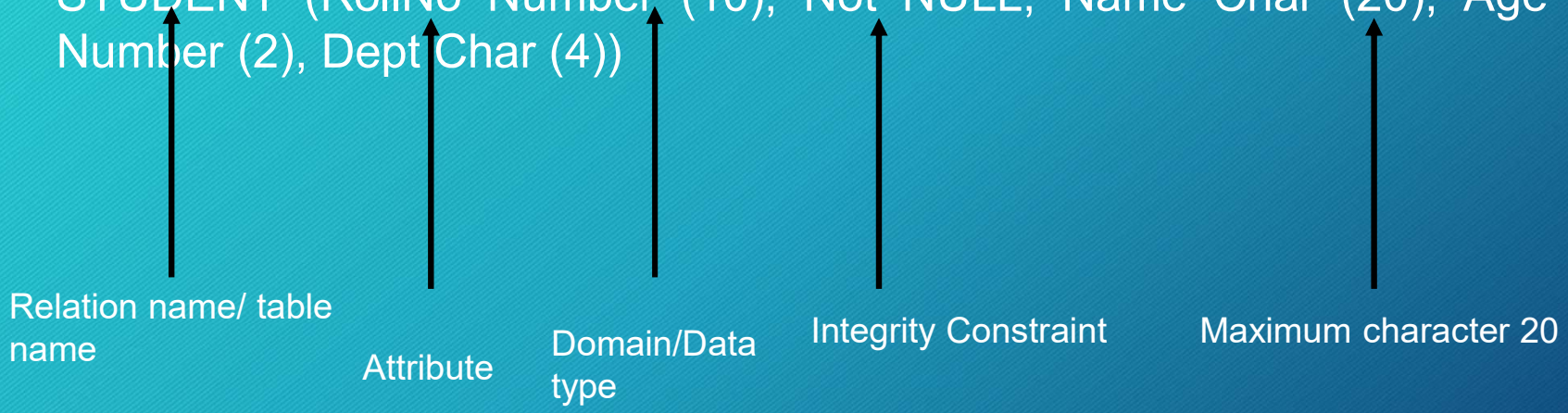
- It is the description of the database
- It is specified during database design
- It is independent of time
- It is the permanent part of the relation
- Intension is not expected to change frequently but changes applied to schema/intension as application requirements change
- Intension is a combination of structure and a set of integrity constraints
- The naming structure consists of the relation name and the names of attributes with its domain name.

Relation Intention and Extension

Intension (Database Schema/Structure)

Example: Intension of STUDENT relation

STUDENT (RollNo Number (10), Not NULL, Name Char (20), Age Number (2), Dept Char (4))



Relation Intention and Extension

Extension (Database Instance/Database State)

- It is the extension of the schema (stores the values)
- The extension of a given relation is the set of tuples appearing in that relation at any given instance (at a particular time)
- The extension varies with time
- The extension changes as tuples are created, destroyed and updated

Relation Intention and Extension

Example: Extension (Database Instance/Database State)
Relation: STUDENT at time t1

Roll No	Name	Age	Dept
1	X	19	CSE

Relation: STUDENT at time t2 (two more records are added)

Roll No	Name	Age	Dept
1	X	19	CSE
2	Y	19	ECE
3	Z	20	ME

Relation Intention and Extension

Intension

- The intension of a given relation is independent of time. It is the permanent part of the relation.
- It corresponds to what is specified in the **relational schema**.
- The intension thus defines all permissible extensions.
- The intension is a combination of two things: a structure and a set of integrity constraints
- The naming structure consists of the relation name plus the names of the attributes (each with its associated domain name)
- The integrity constraints can be subdivided into key constraints, referential constraints & other constraints.
- For example,
- Employee (EmpNo Number (4) Not NULL, EName Char (20), Age Number (2), Dept Char (4))
- This is the intension of Employee relation

Relation Intention and Extension

Extension

The extension of a given relation is the set of tuples appearing in that relation at any given instance. The extension thus varies with time. It changes as tuples are created, destroyed, and updated.

Relation: Employee at time= t1

EmpNo	EmpName	Age	Dept
1001	Jason	23	SD
1002	William	24	HR
1003	Jonathan	28	Fin
1004	Harry	20	Fin

Relation: Employee at time= t2 after adding more records

EmpNo	EmpName	Age	Dept
1001	Jason	23	SD
1002	William	24	HR
1003	Jonathan	28	Fin
1004	Harry	20	Fin
1005	Smith	22	HR
1006	Mary	19	HR
1007	Sarah	23	SD

Difference Between Schema and Instance

Schema	Instance
It is the overall description of the database.	It is the collection of information stored in a database at a particular moment.
Schema is same for whole database.	Data in instances can be changed using addition, deletion, updation.
Does not change Frequently.	Changes Frequently.
Defines the basic structure of the database i.e., how the data will be stored in the database.	It is the set of Information stored at a particular time

Schema

Schema refers to the overall description of any given database. Instance basically refers to a collection of data and information that the database stores at any particular moment.

Example of Schema

EMP_NAME	EMP_ID	EMP_ADDRESS	EMP_CONTACT
--------	------	-----------	-----------

Instance

DBMS Instance

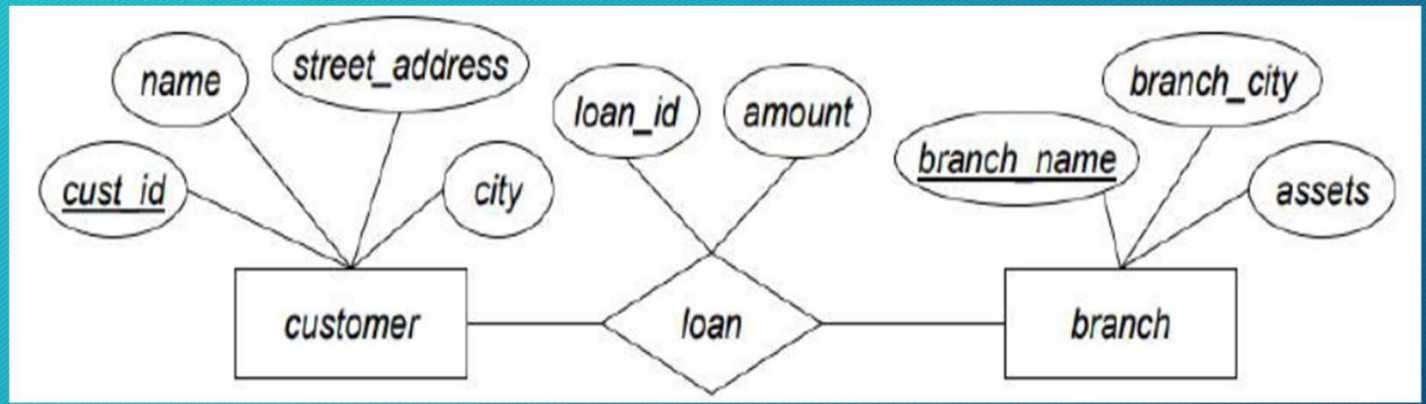
Definition of instance: The data stored in database at a particular moment of time is called instance of database. Database schema defines the attributes in tables that belong to a particular database. The value of these attributes at a moment of time is called the instance of that database.

For example, we have seen the schema of table "employee" above. Let's see the table with the data now. At this moment the table contains two rows (records). This is the the current instance of the table "employee" because this is the data that is stored in this table at this particular moment of time.

EMP_NAME	EMP_ID	EMP_ADDRESS	EMP_CONTACT
Chaitanya	101	Noida	95********
Ajeet	102	Delhi	99********

Define descriptive attributes

- A descriptive attribute or simply a descriptor, describes a non-unique characteristic of an entity instance.
- A set of relationships of similar type is called a relationship set. Like entities, a relationship too can have attributes. These attributes are called descriptive attributes.



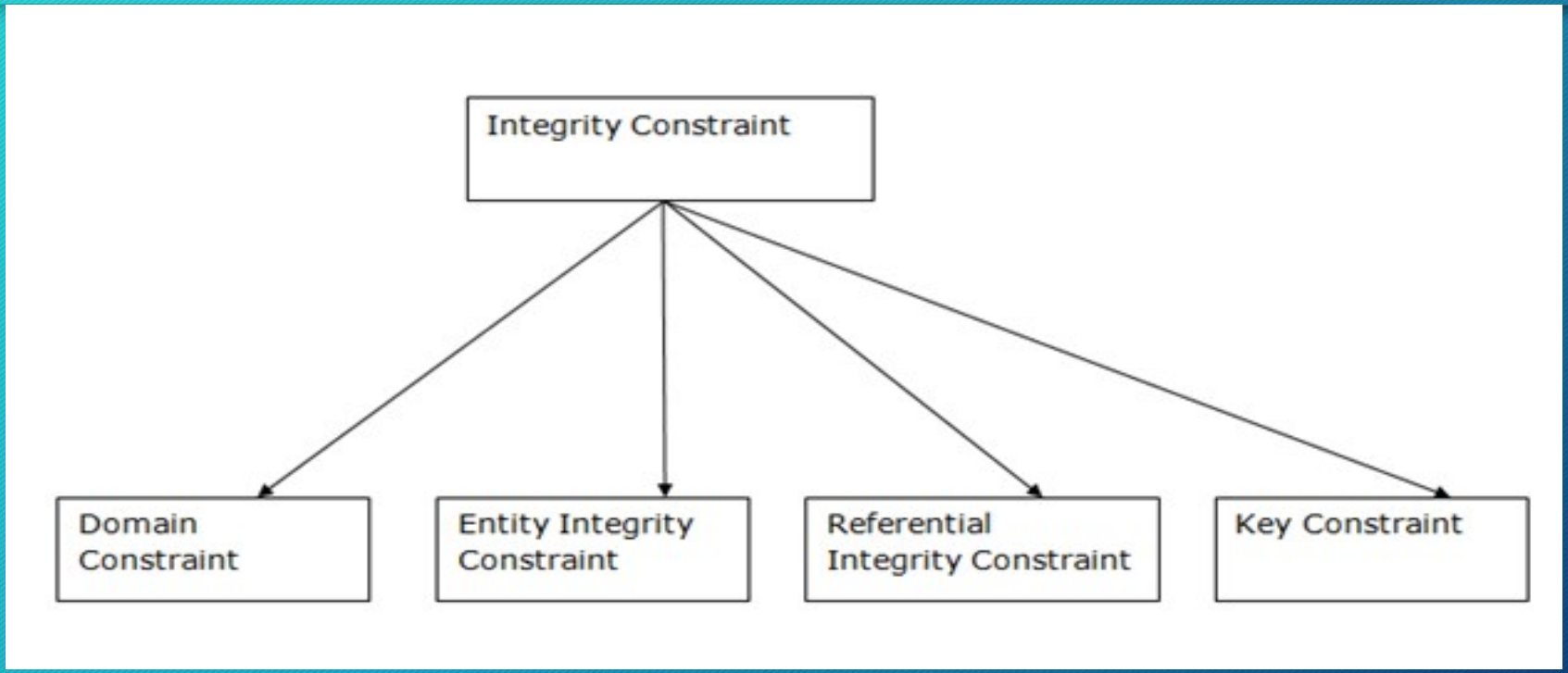
- Represent loans as relationships between customers and branches
- Loan_id and amount are descriptive attributes

Integrity Constraints (Relational Constraints)

- **Integrity constraints are a set of rules. It is used to maintain the quality of information.**
- **Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.**
- **Thus, integrity constraint is used to guard against accidental damage to the database.**

Integrity Constraints (Relational Constraints)

Types of Integrity Constraints



Domain Constraints (Relational Constraints)

1. Domain constraints

- **Domain constraints can be defined as the definition of a valid set of values for an attribute.**
- **The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.**

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed. Because AGE is an integer attribute

Entity Integrity Constraints (Relational Constraints)

2. Entity integrity constraints

- **The entity integrity constraint states that primary key value can't be null.**
- **This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.**
- **A table can contain a null value other than the primary key field.**

Example:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

Referential Integrity Constraints (Relational Constraints)

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables

Foreign Key

- It is an attribute or set of attributes that references to primary key of same table or another table (relation). In foreign key concept we must have one common attribute in two tables otherwise its not possible
- Contains referential integrity

STUDENT
Primary Key (P.K)

COURSE
Foreign Key (F.K)

Roll No	Name	Address
1	A	Delhi
2	B	Mumbai
3	A	Chandigarh

BASE TABLE OR REFERENCED TABLE

Course id	Course Name	Roll No
C1	DBMS	1
C2	NETWORKS	2

REFERENCING TABLE

Referential Integrity Constraints (Relational Constraints)

REFERENCED TABLE

1. **Insert- No violation (whatever entries we will made in base table there is no change to referencing table)**

STUDENT

Primary Key **BASE OR REFERENCED TABLE**

Roll No	Name	Address
1	A	Delhi
2	B	Mumbai
3	A	Chandigarh
4	D	Chandigarh

COURSE

Foreign Key

Cour se id	Course Name	Roll No
1	DBMS	1
2	NETWORKS	2

REFERENCING TABLE

Referential Integrity Constraints (Relational Constraints)

REFERENCED TABLE

2. Delete – One student left the college so its deleted from Base table but the information related to Roll 1 is exist in referencing table and shows that Roll 1 is reading DBMS i.e. integrity is lost So deletion may cause violation.

If we will delete last row (one student has taken admission and immediately left the college but the course has not offered to him) so there is no problem, easily we can delete last entry from base table

Primary Key

STUDENT

Roll No	Name	Address
~~1~~	~~A~~	~~Delhi~~
2	B	Mumbai
3	A	Chandigarh
~~4~~	~~D~~	~~Chandigarh~~

BASE OR REFERENCED TABLE

COURSE

Foreign Key

Course id	Course Name	Roll No
1	DBMS	1
2	NETWORKS	2

REFERENCING TABLE

Referential Integrity Constraints (Relational Constraints)

REFERENCED TABLE

3. Updation- Suppose, we want to change some data, if I will change Roll 2 to Roll 20, we can not change it directly in base table, some problem arises, I updated Roll 2 to Roll 20 in 1st table but in 2nd table it remains as it is Roll No 2, so it leads to data inconsistency. This means now Roll 2 does not exist in 1st table, In base table there is no Roll no 2 but in 2nd table where Roll no 2 came? If you are updating in base table you have to be very careful.

- **So it may cause violation**

Primary Key **STUDENT**

Roll No	Name	Address
1	A	Delhi
2/ 20	B	Mumbai
3	A	Chandigarh
4	D	Chandigarh

BASE OR REFERENCED TABLE

COURSE **Foreign Key**

Course id	Course Name	Roll No
1	DBMS	1
2	NETWORKS	2

REFERENCING TABLE

Referential Integrity Constraints (Relational Constraints)

REFERENCING TABLE

1. **Insert- If we will insert some data in referencing table, then there will be some problem because Roll No 7 is not present in case of Base Table (no student took admission, so how did we give him the course? If there will be data in Base Table as Roll No 5,6,7 then we can offer him a course)**
   - **May cause violation**
2. **Delete- Will not cause any violation (if we will delete any data in referencing table (one student has studied in DBMS, then he left the college, there will be no violation in base table))**

Primary Key **STUDENT**

Roll No	Name	Address
1	A	Delhi
2	B	Mumbai
3	A	Chandigarh
4	D	Chandigarh

BASE OR REFERENCED TABLE

COURSE **Foreign Key**

Course id	Course Name	Roll No
~~1~~	~~DBMS~~	~~1~~
2	NETWORKS	2
3	C++	7

REFERENCING TABLE

Referential Integrity Constraints (Relational Constraints)

REFERENCING TABLE

3. Update- May cause violation

- If you will update Course id, course name then there is no problem. If you will update Roll No in referencing table then there will be violation
- Roll No is updated to 20 in referencing table but Roll No 20 is not present in Base table.
- May cause violation

Primary Key

STUDENT

Roll No	Name	Address
1	A	Delhi
2	B	Mumbai
3	A	Chandigarh
4	D	Chandigarh

BASE OR REFERENCED TABLE

COURSE

Foreign Key

Course id	Course Name	Roll No
1	DBMS	1
2	NETWORKS	20
3	C++	7

REFERENCING TABLE

Key Constraints (Relational Constraints)

4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and not null value in the relational table.
- **Example:**

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1002	Morgan	8th	22

Not allowed. Because all row must be unique

THANK YOU